МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Московский авиационный институт
(Национальный Исследовательский Университет)

Отчёт по лабораторной работе №3

«Управление потоками и синхронизация»

Выполнила: студент группы М8О-203БВ-25
Петрова Эвелина
Вариант: 4

Москва, 2026

1 Цель работы

Приобретение практических навыков в:

- управлении потоками в операционной системе;
- обеспечении синхронизации между потоками.

2 Задание

Составить программу, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы.

Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы.

В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Полученные результаты необходимо объяснить.

Вариант 4: отсортировать массив целых чисел при помощи TimSort.

3 Теоретические сведения

3.1 Процессы и потоки

Процесс — это адресное пространство и ресурсы программы: код, глобальные данные, куча, открытые файлы. Поток — единица планирования внутри процесса: у каждого потока собственные регистры, стек и позиция выполнения, а код, глобальные данные, куча и открытые файлы общие для всех потоков процесса.

Поскольку память общая, потоки обмениваются данными без межпроцессного взаимодействия, но одновременный доступ к одним и тем же данным требует синхронизации (например, мьютекса), иначе возникает состояние гонки.

На одном ядре потоки выполняются конкурентно, по очереди получая процессорное время. На нескольких ядрах потоки могут выполняться параллельно, однако большее число потоков не гарантирует ускорения: создание потоков, переключения и доступ к памяти имеют свою стоимость.

3.2 Интерфейс POSIX Threads

В Linux потоки создаются функциями библиотеки POSIX Threads (`pthread_*`), которые внутри используют системные вызовы `clone`/`clone3` и `futex`.

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start *), void *arg)` — создание потока, который начинает выполнять функцию `start` с аргументом `arg`. При успехе возвращает 0.
- `int pthread_join(pthread_t thread, void **retval)` — ожидание завершения потока.
- `int pthread_mutex_lock(pthread_mutex_t *m)` и `pthread_mutex_unlock` — захват и освобождение мьютекса; в критической секции, защищённой мьютексом, в каждый момент находится не более одного потока.

Сборка выполняется с ключом `-pthread`.

3.3 Алгоритм TimSort

TimSort — гибридный устойчивый алгоритм сортировки, объединяющий сортировку вставками и сортировку слиянием. Он используется в стандартных библиотеках Python и Java и эффективен на реальных данных, которые часто содержат уже упорядоченные фрагменты.

1. Вычисляется параметр `minrun` (от 32 до 64) так, чтобы $n/$`minrun` было равно степени двойки или чуть меньше неё.
2. Массив разбивается на *раны* — естественные неубывающие или строго убывающие подпоследовательности. Убывающие раны разворачиваются.
3. Раны короче `minrun` дополняются следующими элементами до длины `minrun` сортировкой вставками с бинарным поиском.
4. Раны помещаются в стек. Для трёх верхних ранов X, Y, Z поддерживаются инварианты $|X| > |Y| + |Z|$ и $|Y| > |Z|$; при их нарушении соседние раны сливаются. Это делает слияния сбалансированными.
5. При слиянии используется *галопирование*: если один ран выигрывает сравнение 7 раз подряд, позиция следующего элемента ищется экспоненциальным и бинарным поиском, и найденный блок переносится целиком.

Сложность TimSort — $O(n \log n)$ в худшем случае и $O(n)$ на уже отсортированных или развёрнутых данных.

3.4 Ускорение и эффективность

Пусть T_1 — время работы алгоритма в одном потоке, T_p — в p потоках. Тогда

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p},$$

где S_p — ускорение, E_p — эффективность. В идеальном случае $S_p = p$ и $E_p = 1$.

Согласно закону Амдала, если доля f работы выполняется последовательно, то ускорение ограничено:

$$S_p \le \frac{1}{f + \frac{1-f}{p}} < \frac{1}{f}.$$

4 Описание программы

4.1 Структура проекта

```
lab3/
  include/timsort.hpp           объявления TimSort и слияния
  include/parallel_timsort.hpp  объявление параллельной версии
  src/timsort.cpp               последовательная TimSort
  src/parallel_timsort.cpp      работа с потоками
  src/main.cpp                  ключи запуска, данные, замер времени
  scripts/bench.sh              замеры ускорения и эффективности
  scripts/plot.py               построение графиков
  Makefile
```

4.2 Параллельный алгоритм

Пусть задано ограничение в p потоков. Сортировка выполняется в два этапа.

1. **Сортировка частей.** Массив делится на p примерно равных частей. Для каждой части создаётся поток (`pthread_create`), который сортирует её последовательной TimSort. Главный поток ожидает все потоки через `pthread_join`.

2. **Попарные слияния.** Отсортированные части сливаются раундами: в каждом раунде соседние части объединяются парами, и каждое слияние выполняет отдельный поток. Количество частей уменьшается вдвое за раунд ($8 \to 4 \to 2 \to 1$), всего $\lceil \log_2 p \rceil$ раундов. При нечётном числе частей последняя переходит в следующий раунд без изменений.

В любом раунде одновременно работает не более p потоков: на первом этапе ровно p, на втором — не более $p/2$. Главный поток в это время только ожидает и вычислений не выполняет.

4.3 Синхронизация

Каждый рабочий поток обрабатывает свой непересекающийся отрезок массива, поэтому гонки за данные нет и блокировки при сортировке не нужны. Порядок этапов обеспечивает `pthread_join`: раунд слияния начинается только после завершения всех потоков предыдущего раунда.

Общим ресурсом является стандартный вывод: в подробном режиме потоки печатают свои идентификаторы. Чтобы строки разных потоков не перемешивались, вывод защищён мьютексом:

```
pthread_mutex_lock(&print_mutex);
printf("  поток TID=%ld сортирует [%zu, %zu)\n", ...);
pthread_mutex_unlock(&print_mutex);
```

Если `pthread_create` завершается с ошибкой, программа сообщает об этом и выполняет задачу в текущем потоке.

4.4 Ключи запуска

- `-t N` — максимальное число одновременно работающих потоков (обязательный);
- `-n N` — размер массива (по умолчанию 10^6);
- `-g TYPE` — тип данных: `random`, `sorted`, `reversed`, `partial` (отсортированный массив, в котором переставлен 1% элементов);
- `-c` — проверить результат сравнением с `std::sort`;
- `-w SEC` — задержка в рабочих потоках, чтобы их можно было увидеть средствами ОС;
- `-v` — печатать PID процесса и TID потоков;
- `-q` — печатать только время в миллисекундах.

Сборка: `make`. Пример запуска: `./lab3 -t 8 -n 10000000 -c`.

5 Результаты работы

5.1 Проверка корректности

Результат каждой сортировки сравнивается с результатом `std::sort` на копии исходных данных. Программа проверена на всех типах данных, размерах от 0 до 10^7 элементов (включая граничные 1, 63, 64, 65) и числе потоков от 1 до 16 — во всех случаях результат совпал.

5.2 Количество потоков средствами ОС

Программа запускается с задержкой в потоках: `./lab3 -t 4 -n 1000000 -w 10 -v`. Во втором терминале во время работы выполняется `ps -T -p $(pgrep -x lab3)`, которая выводит все потоки процесса:

```
  PID    SPID TTY          TIME CMD
25484   25484 pts/7    00:00:00 lab3
25484   25485 pts/7    00:00:00 lab3
25484   25486 pts/7    00:00:00 lab3
25484   25487 pts/7    00:00:00 lab3
25484   25488 pts/7    00:00:00 lab3
```

У процесса пять потоков: главный (его SPID совпадает с PID) и четыре рабочих, TID которых совпадают с напечатанными программой. Это соответствует ограничению `-t 4`.

6 Исследование ускорения и эффективности

6.1 Методика

Замеры выполнены на процессоре AMD Ryzen 7 7735HS (8 ядер, 16 логических процессоров) в WSL2 (Ubuntu). Время измеряется только для сортировки, без генерации данных. Каждая конфигурация запускалась 3 раза, бралось лучшее время. Исследованы три типа данных (случайные, почти отсортированные, в обратном порядке), размеры 10^5, 10^6, 10^7 и число потоков 1, 2, 4, 8, 16. Замеры выполняет скрипт `scripts/bench.sh`.

6.2 Результаты

Таблица 1: Время, ускорение и эффективность

Данные	n	p	T_p, мс	S_p	E_p
random	100 000	1	11.5	1.00	1.00
random	100 000	2	7.8	1.48	0.74
random	100 000	4	5.2	2.21	0.55
random	100 000	8	5.1	2.24	0.28
random	100 000	16	6.4	1.79	0.11
random	1 000 000	1	136.8	1.00	1.00
random	1 000 000	2	81.8	1.67	0.84
random	1 000 000	4	51.4	2.66	0.67
random	1 000 000	8	37.1	3.69	0.46
random	1 000 000	16	36.6	3.74	0.23

Данные	n	p	T_p, мс	S_p	E_p
random	10 000 000	1	1590.6	1.00	1.00
random	10 000 000	2	913.0	1.74	0.87
random	10 000 000	4	576.9	2.76	0.69
random	10 000 000	8	393.1	4.05	0.51
random	10 000 000	16	393.6	4.04	0.25
partial	100 000	1	2.3	1.00	1.00
partial	100 000	2	1.4	1.67	0.83
partial	100 000	4	1.9	1.22	0.30
partial	100 000	8	2.2	1.04	0.13
partial	100 000	16	3.8	0.61	0.04
partial	1 000 000	1	26.2	1.00	1.00
partial	1 000 000	2	18.6	1.41	0.71
partial	1 000 000	4	13.5	1.95	0.49
partial	1 000 000	8	14.4	1.82	0.23
partial	1 000 000	16	14.6	1.80	0.11
partial	10 000 000	1	203.0	1.00	1.00
partial	10 000 000	2	175.5	1.16	0.58
partial	10 000 000	4	153.9	1.32	0.33
partial	10 000 000	8	140.4	1.45	0.18
partial	10 000 000	16	134.0	1.51	0.10
reversed	100 000	1	1.1	1.00	1.00
reversed	100 000	2	1.3	0.91	0.45
reversed	100 000	4	1.4	0.79	0.20
reversed	100 000	8	2.1	0.54	0.07
reversed	100 000	16	4.0	0.28	0.02
reversed	1 000 000	1	12.0	1.00	1.00
reversed	1 000 000	2	10.5	1.14	0.57
reversed	1 000 000	4	11.0	1.09	0.27
reversed	1 000 000	8	11.8	1.02	0.13
reversed	1 000 000	16	13.2	0.91	0.06
reversed	10 000 000	1	159.0	1.00	1.00
reversed	10 000 000	2	116.7	1.36	0.68
reversed	10 000 000	4	108.7	1.46	0.37
reversed	10 000 000	8	126.7	1.25	0.16
reversed	10 000 000	16	122.4	1.30	0.08

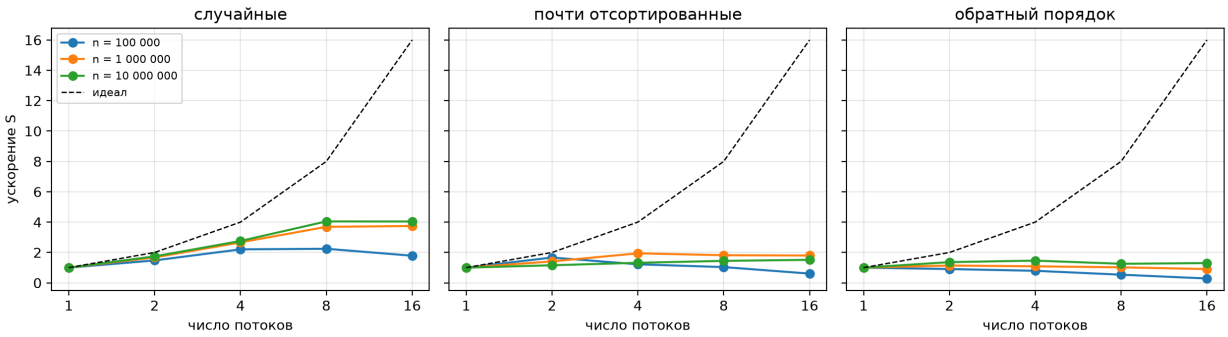


Рис. 1: Зависимость ускорения от числа потоков

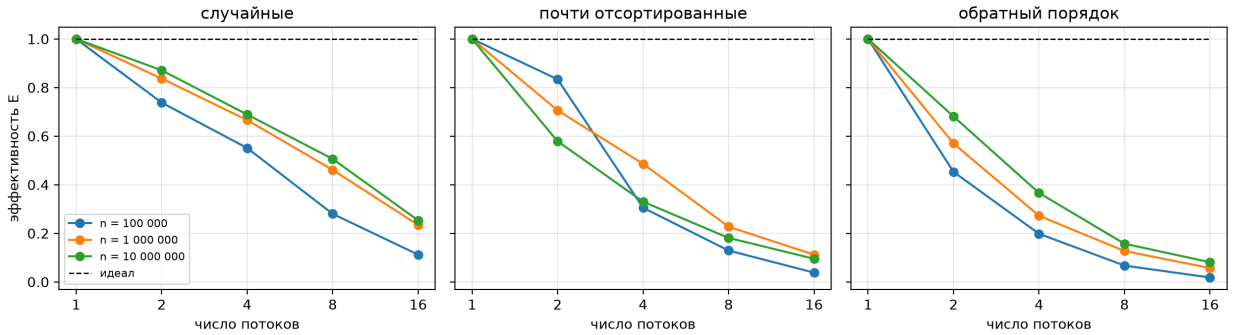


Рис. 2: Зависимость эффективности от числа потоков

6.3 Анализ

Случайные данные. Здесь достигается наибольшее ускорение: около 4 на 10^7 элементов при 8 потоках. Основная работа — сортировка частей со сложностью $O\left(\frac{n}{p}\log\frac{n}{p}\right)$ — полностью распараллеливается. При этом ускорение растёт медленнее числа потоков, а эффективность падает (0,87 при 2 потоках, 0,51 при 8), что объясняется законом Амдала: последний раунд слияния обрабатывает весь массив в одном потоке, и его время $O(n)$ не уменьшается с ростом p. Чем больше потоков, тем большую долю общего времени составляет эта последовательная часть.

Переход от 8 к 16 потокам практически не даёт выигрыша. Процессор имеет 8 физических ядер; 16 логических процессоров получаются за счёт SMT (два потока на ядро), а два потока одного ядра делят его вычислительные блоки и кэш. Кроме того, слияние в основном копирует данные и упирается в пропускную способность памяти, которая общая для всех ядер.

Размер массива. На 10^5 элементов ускорение не превышает 2,2, а при 16 потоках время растёт. Сама сортировка занимает около 10 мс, и накладные расходы на создание потоков и дополнительные раунды слияния становятся сравнимы с полезной работой. С ростом n эти расходы распределяются на большую работу, и ускорение увеличивается.

Почти отсортированные и обратные данные. Ускорение здесь мало (не более 2), а на малых массивах параллельная версия медленнее последовательной. Это следствие сильной стороны TimSort: на таких данных он находит длинные раны и работает почти за $O(n)$ — например, 10^7 элементов в обратном порядке сортируются в одном потоке за 159 мс против 1591 мс для случайных. Вычислений, которые можно разделить между потоками, почти нет, а остающаяся работа — копирование и слияние — ограничена пропускной способностью памяти. В случае обратного порядка разбиение на части даже вредит: каждая часть разворачивается отдельно, после чего части приходится сливать, хотя последовательная версия обошлась бы одним разворотом всего массива.

Колебания отдельных значений (например, для обратных данных при 8 и 16 потоках) связаны с малым временем работы (около 100 мс) и влиянием фоновой нагрузки Windows и WSL.

Выводы

В ходе лабораторной работы была реализована параллельная сортировка массива целых чисел алгоритмом TimSort с использованием POSIX Threads. Число одновременно работающих потоков ограничивается ключом запуска `-t`, а сами потоки видны сред-

ствами ОС (`ps -T`). Корректность результата проверена сравнением со стандартной сортировкой.

Параллельная схема «сортировка частей + попарные слияния» не требует блокировок при обработке данных, так как потоки работают с непересекающимися участками массива, а порядок этапов задаётся ожиданием потоков через `pthread_join`. Мьютекс используется только для общего ресурса — стандартного вывода.

Исследование показало, что ускорение зависит как от числа потоков, так и от входных данных. На больших случайных массивах получено ускорение около 4 при 8 потоках; дальнейший рост ограничен последовательным последним слиянием (закон Амдала), числом физических ядер и пропускной способностью памяти. На малых массивах и на частично упорядоченных данных, которые TimSort и так обрабатывает почти за линейное время, накладные расходы на потоки сравнимы с полезной работой, и распараллеливание выгоды не даёт. Таким образом, увеличение числа потоков оправдано только при достаточно большом объёме вычислительной работы.